

MethylCentroid (Statistical background)

COMPUTATIONAL TEAM - EPI METHYL ANALYTICS

Summary

MethylCentroid is a Python-based tool for computing methylation centroids from genomic data using an efficient position-aligned averaging approach with outlier removal. It aggregates methylation profiles from multiple samples (stored in HDF5 format) to create a representative centroid that summarizes group-level methylation statistics, identifying and removing outlier samples to ensure robustness.

A **basic centroid** is a synthetic sample, similar in structure to any other sample, that is, with methylation data as:

- pos: position
- mC: methylated reads
- uC: unmethylated reads
- tnc: bitmapped field with information about the trinucleotide and strand.

An **extended centroid** is a complete representation of a group of samples, storing each position and the statistical distribution of its methylation levels.

- Pos: position
- SI: $\text{sum}(ml)$ where $ml = \frac{mc}{mc+uC}$ for samples with $mC + uC > 0$
- SI2: $\text{sum}(ml^2)$
- N: number of valid samples (added to SI).
- Log_x_sum: $\text{sum}(\log(\text{sum}(ml)))$
- Log_1_minus_x_sum:
- tnc: same as before

The **extended centroid** contains enough information to calculate the sufficient statistics for the Beta distribution at each position.

The **basic centroid**, with the use of SI, SI2, and N, should be used for calculating divergences or distances between samples.

The extended centroid should be used to compare against another extended centroid, because it presents the whole group of samples.

The statistical framework includes:

- Position alignment across samples using dense matrix operations,
- Centroid computation via averaging of methylated (mC) and unmethylated (uC) counts,
- Iterative outlier detection and removal using hypothesis testing,
- Coverage-based filtering for reliable positions,
- Group-level methylation statistics computation (e.g., mean, variance, entropy),
- GPU acceleration for large-scale computations, and
- Batch processing for multiple chromosomes and contexts.

This report details the mathematical formulations, assumptions, and implementation of these methods. It draws directly from the project's core modules:

- `methyl_centroid.py` — main centroid computation class,
- `centroid_cli.py` — command-line interface for single chromosome/context processing,
- `gpu_acceleration.py` — GPU utilities for accelerated operations,
- `genomic_position_aligner.py` — position alignment and data aggregation (via `PositionAligner` class),
- `methylation_analysis.py` — auxiliary methylation feature computation (e.g., entropy, divergence).

1. Introduction

To study DNA methylation across groups of samples, we propose a method to summarize their methylation levels into a single, representative profile. This is where the concept of a “centroid” becomes useful.

Methylation data in genomics is typically represented as counts of methylated (mC) and unmethylated (uC) reads at each genomic position, with total coverage ($mC + uC$) indicating data reliability. These counts can be aggregated across samples to form a **centroid**, which is a group-level summary storing averaged mC and uC values, along with position coordinates and additional data (e.g., trinucleotide code, tnc).

For each genomic position in a specific chromosome and context (e.g., CG, CHG, CHH), the **MethylCentroid** tool aligns data from multiple samples, filters for sufficient coverage, computes averages, and iteratively removes outliers based on statistical tests. This ensures the centroid represents a homogeneous group, excluding anomalous samples that may arise from technical artifacts or biological variability. (Tukey 1977)

The position-aligned approach in **MethylCentroid** processes data through a structured sequence of steps:

- **Aligning positions across samples.** Uses a dense matrix to handle varying position sets, ensuring all samples contribute to common positions.
- **Filtering positions with sufficient coverage** ($mC + uC \geq \text{min_coverage}$), default 4. Positions with low coverage yield unstable estimates and can introduce noise; enforcing a minimum improves reliability.
- **Performing outlier detection via hypothesis testing.** Iteratively identifies and removes samples that deviate significantly from the group distribution.

GPU acceleration (via `CuPy`) is used for vectorized operations when available, falling back to CPU (`NumPy`) otherwise, with multiprocessing for efficiency.

2. Centroid Data Structure and Preprocessing

Before analyzing methylation data, we need to organize it into a common format. This means aligning sample data and filtering it so that only reliable information is used.

Centroids are stored in `HDF5` files, each containing datasets that summarize methylation count information for multiple genomic positions:

- `pos`: Genomic coordinates (`uint32`) of the CpG or other sites.
- `mC, uC`: Averaged methylated and unmethylated counts across samples (`uint32`).
- `tnc`: Trinucleotide code (`uint8`) which encodes the original tnc and strand.
- `sample_count`: Number of samples contributing to the centroid.

Common genomic positions between samples are identified using the `PositionAligner` class, which dynamically expands the position range to encompass all input data via set union and indexing. Positions are then filtered to remove those with insufficient coverage ($mC + uC < \text{min_coverage}$) or invalid values (e.g., `NaN`). This ensures that the centroid is computed only from reliable, comparable data. (Casella 2002)

The `PositionAligner` uses a dense matrix representation where rows correspond to aligned positions and columns to samples. Missing data at non-overlapping positions is implicitly handled by zero-filling or exclusion during averaging. This structure **condenses methylation counts into aggregates**, reducing memory usage and enabling fast centroid computation for millions of positions.

3. Statistical Methods for Centroid Computation

Once the data is aligned, we calculate an average methylation signal across all samples. This average becomes our group-level summary at each genomic position.

For each position, the centroid is computed as the sum of mC and uC across samples. (Johnson 1995)

4. Position Alignment and Averaging

Not all samples cover the same genomic positions, so we need to align them. This ensures we compare methylation levels at the same sites across all samples.

The `PositionAligner` class aligns positions across samples by:

- Dynamically determining the global position range: $min_pos = \min(all_sample_mins)$, $max_pos = \max(all_sample_maxs)$.
- Creating a dense matrix M (shape: $num_positions \times num_samples$) where $M[i, j] = mC$ or uC for sample j at position i , or 0 if missing.
- Computing coverage: $C[i, j] = mC[i, j] + uC[i, j]$.
- Filtering valid positions i where $C[i, j] \geq min_coverage$ (default 4) for all samples j .

The centroid averages are then:

$$centroid_mC[i] = \frac{\sum mC[i, :]}{sample_count}$$

$$centroid_uC[i] = \frac{\sum uC[i, :]}{sample_count}$$

Where $sample_count$ is the number of active samples after outlier removal.

This averaging assumes independence across positions and samples, providing a robust group summary. (Casella 2002)

5. Outlier Detection and Removal

Sometimes, a sample behaves very differently from the rest. To avoid distortion, we automatically detect and remove such “outlier” samples from the group summary.

Outliers are detected iteratively using **Jeffreys divergence** to measure the statistical dissimilarity between each sample and the current group centroid. (Jeffreys 1946) For each iteration:

- The centroid is computed from all active (non-removed) samples.
- For every individual sample, the **Jeffreys divergence** is computed between its methylation profile and the centroid.
- The most extreme sample is identified as the one with maximum **divergence**.
- A **p-value is computed using a normal distribution approximation** (i.e., Z-score), based on the distribution of all divergence values in the current iteration.
- If the smallest p-value is below α , default 0.05, the corresponding sample is removed as an outlier.

Procedure:

- Start with the full set of samples to compute the initial centroid.
- At each iteration:
 - Compute Jeffreys divergence for all active samples.
 - Calculate the mean divergence (μ_{div}) and standard deviation (σ_{div}) of the divergence values.
 - Determine the p-value of the most extreme divergence as: $p = 2(1 - \phi(|Z|))$ where $z = \frac{obs - \mu}{\sigma}$ and ϕ is the standard normal CDF.
 - Remove the sample with the lowest p-value if $p < \alpha$.
- Repeat up to *max_iterations* (default: 10), or stop early if:
 - No more significant outliers are found (all $p \geq \alpha$), or
 - The number of remaining samples falls below *min_samples*, default: 3.

This process ensures that only statistically consistent samples contribute to the final centroid. It replaces earlier resampling-based (bootstrap) approaches with a faster and deterministic parametric method, while maintaining robust detection of outliers. (Efron 1993)

6. Methylation Statistics and Features

After building the centroid, we compute summary statistics like the average methylation ratio, variability, and entropy. This helps describe the behavior of the group.

Post-centroid computation, group-level statistics are calculated:

- Mean methylation ratio: $\mu = \text{mean}(\frac{mC}{mC+uC})$ across positions. Here, μ refers to the mean methylation ratio (not to be confused with μ_{div} in divergence analysis).
- Variance: $Var = \text{var}(\frac{mC}{mC+uC})$ Var denotes variance across positions.
- Shannon entropy: $H = -p \log_2(p) - (1 - p) \log_2(1 - p)$, where p is the methylation ratio, averaged over positions.
- Divergence measures (e.g., Jeffreys or Jensen-Shannon) for sample-to-centroid comparison, though primarily used in outlier detection.

These are computed using vectorized operations, with safe clipping (e.g., $p \in [10^{-10}, 1 - 10^{-10}]$) to avoid numerical issues.

- GPU acceleration is employed for large arrays via functions like `gpu_ops.mean_operation`, `divide_operation`, etc.

7. Batch Processing and Global Statistics

Large projects often involve many chromosomes and contexts. Batch processing allows us to repeat the centroid-building procedure efficiently across all of them.

For multiple chromosomes/contexts, batch processing aggregates results:

- Process each combination independently.
- Collect global statistics: total outliers removed, final sample counts, centroid paths.
- Save batch summary in JSON, including per-combination methylation stats (alignment stats, group means/variances).

No formal multiple testing correction is applied, as outlier detection is per-group; however, global outlier counts provide an overview of data quality.

8. Grouping Positions (Optional)

Nearby genomic positions with similar methylation levels can be grouped together. This helps us define broader regions that may be biologically relevant.

While not core to centroid computation, positions can be grouped post hoc into contiguous regions based on coverage or methylation levels, using *max_gap* (e.g., 100 bp) for merging.

Procedure:

1. Sort valid positions.
2. Merge if $pos[i + 1] - pos[i] \leq max_gap$
 - Output as (start, end) tuples in CSV/JSON.

9. Implementation Details and Performance

The tool is designed to run efficiently on large datasets by using GPU acceleration when available and switching to CPU otherwise.

- **GPU/CPU Switching:** Operations use CuPy when GPU detected (via `pynvml` or fallback), with automatic transfer (`get_gpu_array`). Fallback to NumPy for CPU, with `ThreadPoolExecutor` for parallelism (`n_jobs=auto`).
- **Edge Cases:** Handles small sample sizes ($N < min_samples$), extreme counts (clipping), and memory constraints (`batch_size optimization`).")
- **Outputs:** HDF5 centroids, JSON statistics (per-combination and batch), TXT summaries, with optional verbose logging.
- **Version:** Current release: `v0.2.0` — implementing position-aligned centroid computation with outlier removal for efficient methylation summarization.

10. Assumptions and Limitations

As with any statistical method, our approach relies on a few assumptions about the data and may have limitations in certain edge cases.

- **Distributional Assumption:** Methylation counts (mC , uC) are assumed non-negative integers, with ratios following a Beta-binomial-like distribution implicitly via averaging.

Independence across positions and samples is assumed. (Sanchez 2023) For outlier detection, divergence statistics are further approximated as normal to compute p-values.

- **Bootstrap Approximations:** Outlier p-values rely on sufficient bootstrap samples; accuracy may degrade with very small N or highly skewed data.
- **Coverage and Alignment:** Assumes positions are sorted and unique per sample; dense matrix alignment may consume memory for sparse data.
- **Environment Restrictions:** Runs without internet access, limited to pre-installed libraries (NumPy, SciPy, CuPy, h5py, pysam). GPU detection requires `pynvml`.

Conclusion

MethylCentroid's statistical pipeline enables efficient and robust computation of methylation centroids, incorporating outlier removal and group statistics to ensure high-quality summaries. The position-aligned design and GPU/CPU hybrid processing allow large-scale datasets to be processed quickly while maintaining statistical rigor.

Potential future enhancements include advanced divergence metrics, covariate integration, and support for weighted averaging. For details on running analyses, see the `centroid_cli.py` main function or the **MethylCentroid** class implementation.

References

- Casella, G., & Berger, R. L. 2002. *Statistical Inference*. (2nd ed.). Duxbury.
- Efron, B., & Tibshirani, R. J. 1993. "An Introduction to the Bootstrap." *Chapman and Hall/CRC*.
- Johnson, N. L., Kotz, S., & Balakrishnan, N. 1995. *Continuous Univariate Distributions*. Volume 2. Wiley.
- Lin, J. 1977. "Divergence Measures Based on the Shannon Entropy." *IEEE Transactions on Information Theory*.
- Sanchez, R., Mackenzie, S. 2023. "On the thermodynamics of DNA methylation process." *Scientific Report* 13, 8914 (2023). <https://doi.org/10.1038/s41598-023-35166-9>.
- Tukey, J. W. 1977. *Exploratory Data Analysis*. Addison-Wesley.

